

Functional Programming

Patrick Bahr

Parser Combinators

Based on slides by Jesper Bengtson

Last week

- Programming with monads
(a.k.a. “railway-oriented programming”)
 - ▶ Propagate errors (and error messages)
 - ▶ Maintain and update internal state
- Computation expressions

This week

- Context-free grammars & parsing
- Parser for arithmetic expressions
- Parser combinators

Part I

Context-Free Grammars

Context-free grammars

Context-free grammars (CFGs) can be used to describe languages, e.g. programming languages or data formats

Ingredients of CFGs:

- Terminal symbols
- Non-terminal symbols
- Production rules

Each production rule rewrites a non-terminal symbol to strings of non-terminal and terminal symbols

Context-free grammars

Context-free grammars (CFGs) can be used to describe languages, e.g. programming languages or data formats

Ingredients of CFGs:

- Terminal symbols
- Non-terminal symbols
- Production rules

Example

$$\begin{array}{l} A \rightarrow A + A \\ A \rightarrow x \end{array}$$

Each production rule rewrites a non-terminal symbol to strings of non-terminal and terminal symbols

Context-free grammars

Context-free grammars (CFGs) can be used to describe languages, e.g. programming languages or data formats

Ingredients of CFGs:

- Terminal symbols
- Non-terminal symbols
- Production rules

Example

$$\begin{array}{l} \underline{A} \rightarrow \underline{A} + \underline{A} \\ \underline{A} \rightarrow x \end{array}$$

Each production rule rewrites a non-terminal symbol to strings of non-terminal and terminal symbols

Context-free grammars

Context-free grammars (CFGs) can be used to describe languages, e.g. programming languages or data formats

Ingredients of CFGs:

- Terminal symbols
- Non-terminal symbols
- Production rules

Example

$$\begin{array}{l} \underline{A} \rightarrow \underline{A} + \underline{A} \\ \underline{A} \rightarrow \underline{x} \end{array}$$

Each production rule rewrites a non-terminal symbol to strings of non-terminal and terminal symbols

Context-free grammars

Context-free grammars (CFGs) can be used to describe languages, e.g. programming languages or data formats

Ingredients of CFGs:

- Terminal symbols
- Non-terminal symbols
- Production rules

Example

A → A + A
A → x

Short hand notation (a.k.a. BNF)

A ::= A + A | x

Each production rule rewrites a non-terminal symbol to strings of non-terminal and terminal symbols

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variable names} \rangle$

$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$$\left. \begin{array}{l} N ::= \langle \textit{integers} \rangle \\ V ::= \langle \textit{variable names} \rangle \end{array} \right\} \begin{array}{l} \text{We omit explicit production} \\ \text{rules for } N \text{ and } V. \end{array}$$
$$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$$

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$$\left. \begin{array}{l} N ::= \langle \textit{integers} \rangle \\ V ::= \langle \textit{variable names} \rangle \end{array} \right\} \begin{array}{l} \text{We omit explicit production} \\ \text{rules for } N \text{ and } V. \end{array}$$

$$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$$

In the notation you know from *Discrete Mathematics*:

$$A \rightarrow N$$

$$A \rightarrow V$$

$$A \rightarrow A + A$$

$$A \rightarrow A * A$$

$$A \rightarrow (A)$$

$$A \rightarrow - A$$

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variable names} \rangle$

$A ::= N \mid V \mid A \text{ + } A \mid A \text{ * } A \mid (A) \mid \text{ - } A$

Terminals

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variable names} \rangle$

$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$

Non-terminals

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variable names} \rangle$

$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$

Examples:

Which of these are generated by the grammar?

$x * (1 + y)$

$5 + -y$

$x + (5 * 2)$

$x (1 + y)$

$5 + *y$

$x + ()$

Grammar of arithmetic expressions

Grammar for arithmetic expressions with variables

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variable names} \rangle$

$A ::= N \mid V \mid A + A \mid A * A \mid (A) \mid - A$

Examples:

Which of these are generated by the grammar?

$x * (1 + y)$



$x (1 + y)$



$5 + -y$



$5 + *y$



$x + (5 * 2)$



$x + ()$



Abstract syntax trees

We can represent an expression described by this grammar by a data type called an **abstract syntax tree** (AST) or **expression tree**.

```
type aexp =  
| N of int  
| V of string  
| Add of aexp * aexp  
| Mul of aexp * aexp  
| Neg of aexp
```


Abstract syntax trees

We can represent an expression described by this grammar by a data type called an **abstract syntax tree** (AST) or **expression tree**.

```
type aexp =  
| N of int  
| V of string  
| Add of aexp * aexp  
| Mul of aexp * aexp  
| Neg of aexp
```

For example:

```
Add (N 3, Mul (N 5, V "x"))
```

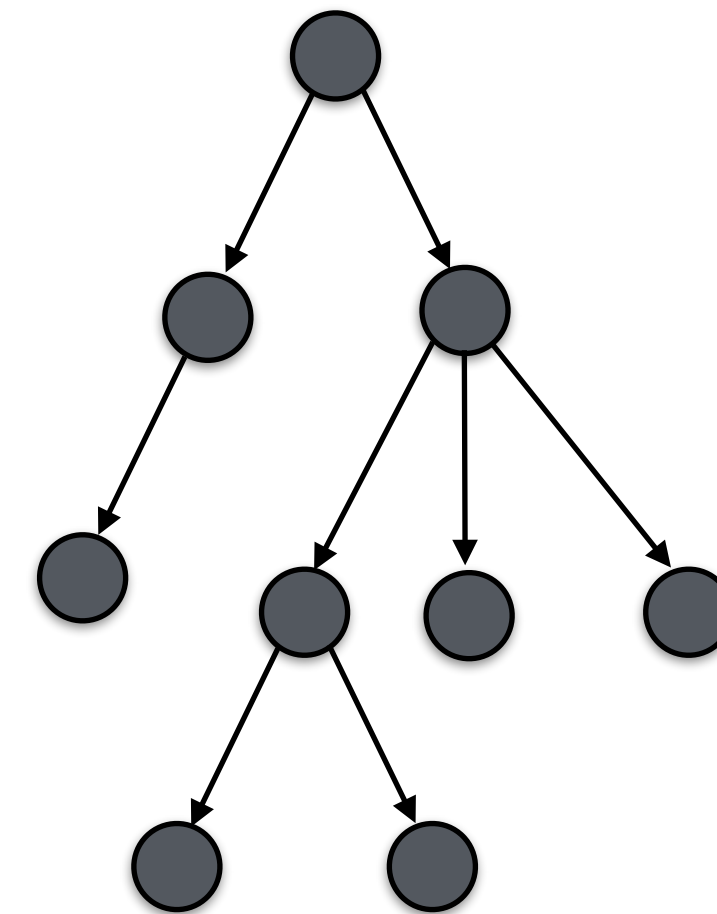
represents the expression

```
3 + (5 * x)
```

Parsers

In general, parsers translate unstructured input (text, binary, bit streams ...) into a data structure

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec elementum sem id lectus efficitur convallis. Cras diam diam, elementum in libero sed, egestas dictum augue. Ut mauris lacus, interdum et dui sit amet, vestibulum mattis sapien. Nulla quis nibh urna. Maecenas sit amet nisi et est vehicula sagittis. Mauris a magna efficitur, varius lectus eget, imperdiet massa. Vivamus sodales nisl eget risus eleifend, at sollicitudin odio aliquet. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Lorem ipsum dolor sit amet, consectetur adipiscing elit.



unstructured input

abstract syntax tree

Parsers

In general, parsers translate unstructured input (text, binary, bit streams ...) into a data structure



unstructured input

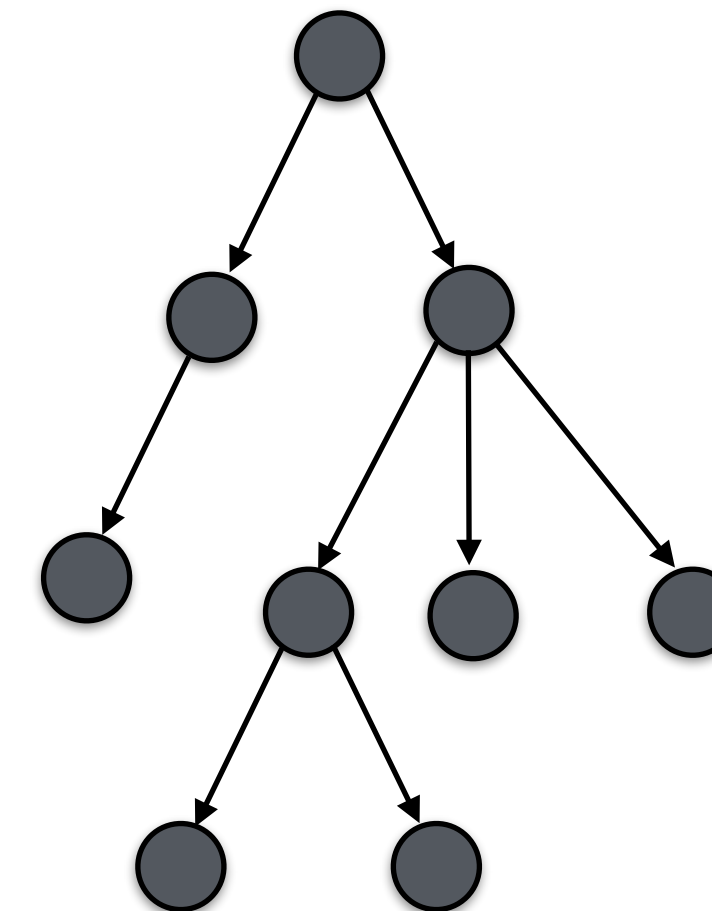
abstract syntax tree

Parsers

In general, parsers translate unstructured input (text, binary, bit streams ...) into a data structure

```
01100110 11001110 01011110 10111001
10010101 01110111 11100110 11010101
11100001 01100001 00101110 11110100
00010100 01111111 00110000 00011001
00011000 01000110 10010010 01101010
10000011 00110110 01010011 01010101
11010010 10110100 11100000 00000010
11100010 11010000 11001100 01101001
10001111 01111111 01110111 00101000
00100011 01011110 10111001 00100010
11100000 11110110 00011010 10010100
00101010 00110111 11110001 00100001
```

parse



unstructured input

abstract syntax tree

The job of a parser

Given a grammar

and

an AST data type

```

N ::= <integers>
V ::= <variables>

A ::= N      | V
    | A + A  | A * A
    | -A     | ( A )
  
```

```

type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
  
```

turn "3 + 5 * x" into `Add (N 3, Mul (N 5, V "x"))`

The job of a parser

Given a grammar

and

an AST data type

```

N ::= <integers>
V ::= <variables>

A ::= N      | V
    | A + A  | A * A
    | -A     | ( A )
  
```

```

type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
  
```

turn "3 + 5 * x" into `Add (N 3, Mul (N 5, V "x"))`

Precedence

Given a grammar

N ::= <integers>		
V ::= <variables>		
A ::= N		V
A + A		A * A
-A		(A)

How do we ensure that

3 + 5 * x

parses into

Add (N 3, Mul (N 5, V "x"))

but not

Mul (Add (N 3, N 5), V "x")

?

Precedence

Given a grammar

$N ::= \langle \textit{integers} \rangle$	
$V ::= \langle \textit{variables} \rangle$	
$A ::= N$	$ V$
$ A + A$	$ A * A$
$ -A$	$ (A)$

How do we ensure that

$3 + (5 * x)$ **this**

parses into

Add (N 3, Mul (N 5, V "x"))

but not

Mul (Add (N 3, N 5), V "x")

?

Precedence

Given a grammar

$N ::= \langle \textit{integers} \rangle$	
$V ::= \langle \textit{variables} \rangle$	
$A ::= N$	$ V$
$ A + A$	$ A * A$
$ -A$	$ (A)$

How do we ensure that

not this $(3 + 5) * x$

parses into

$\text{Add} (N\ 3, \text{Mul} (N\ 5, V\ "x"))$

but not

$\text{Mul} (\text{Add} (N\ 3, N\ 5), V\ "x")$

?

Precedence

Given a grammar

$N ::= \langle \textit{integers} \rangle$	
$V ::= \langle \textit{variables} \rangle$	
$A ::= N$	$ V$
$ A + A$	$ A * A$
$ -A$	$ (A)$

How do we ensure that

not this $(3 + 5) * x$

parses into

$\text{Add} (N\ 3, \text{Mul} (N\ 5, V\ "x"))$

but not

$\text{Mul} (\text{Add} (N\ 3, N\ 5), V\ "x")$

Problem: A naive parser would read the input left-to-right and apply the production rules in order. That would yield the wrong result!

Precedence

Problem: A naive parser would read the input left-to-right and apply the production rules in order. That would yield the wrong result!

N	::=	<integers>									
V	::=	<variables>									
A	::=	<table border="0"> <tr> <td>N</td> <td> </td> <td>V</td> </tr> <tr> <td> A + A</td> <td> </td> <td>A * A</td> </tr> <tr> <td> -A</td> <td> </td> <td>(A)</td> </tr> </table>	N		V	A + A		A * A	-A		(A)
N		V									
A + A		A * A									
-A		(A)									

Solution:

We need to change this grammar so that operator precedence is enforced by it!

Grammar of arithmetic expressions

Revised grammar:

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$

This is better.
We ensure that precedence
order is maintained.

Grammar of arithmetic expressions

Revised grammar:

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$

This is better.
We ensure that precedence
order is maintained.

Let's try to parse
this expression:

$3 + 5 * x$

Grammar of arithmetic expressions

Revised grammar:

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$

3 + 5 * x

Grammar of arithmetic expressions

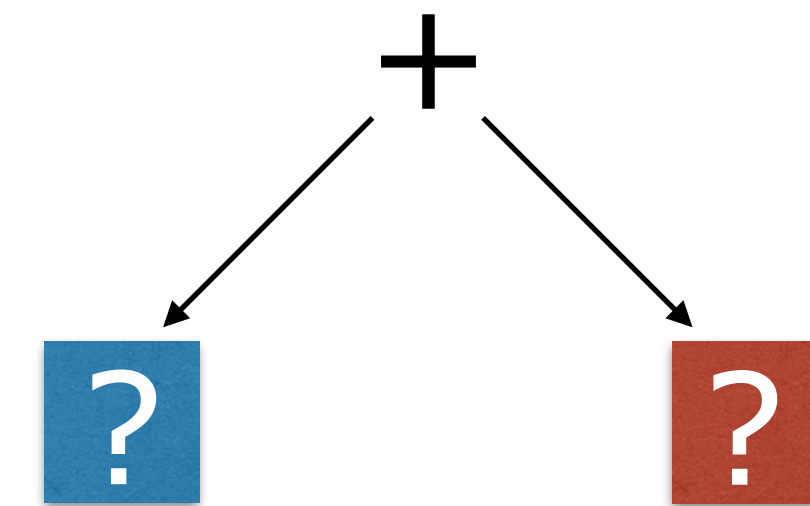
Revised grammar:

$N ::= \langle \textit{integers} \rangle$
 $V ::= \langle \textit{variables} \rangle$

$T ::= \boxed{T} + \boxed{T} \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



$\boxed{3} + \boxed{5 * x}$

Grammar of arithmetic expressions

Revised grammar:

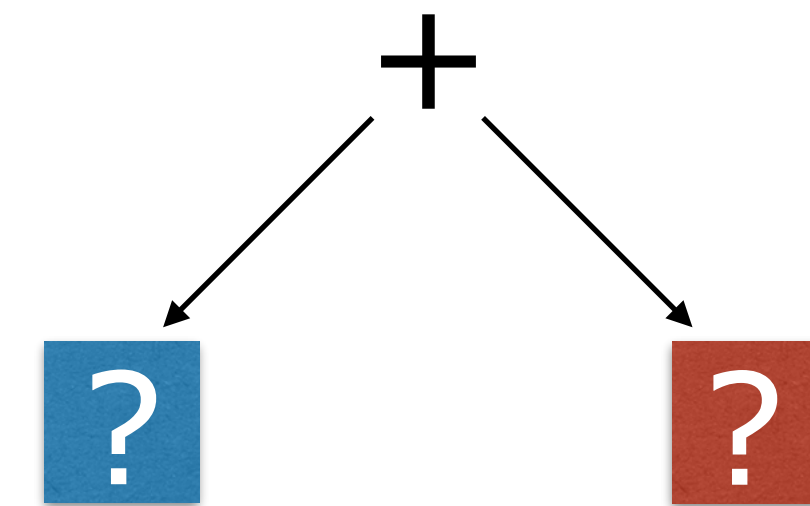
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid \boxed{P}$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



$\boxed{3} + \boxed{5 * x}$

Grammar of arithmetic expressions

Revised grammar:

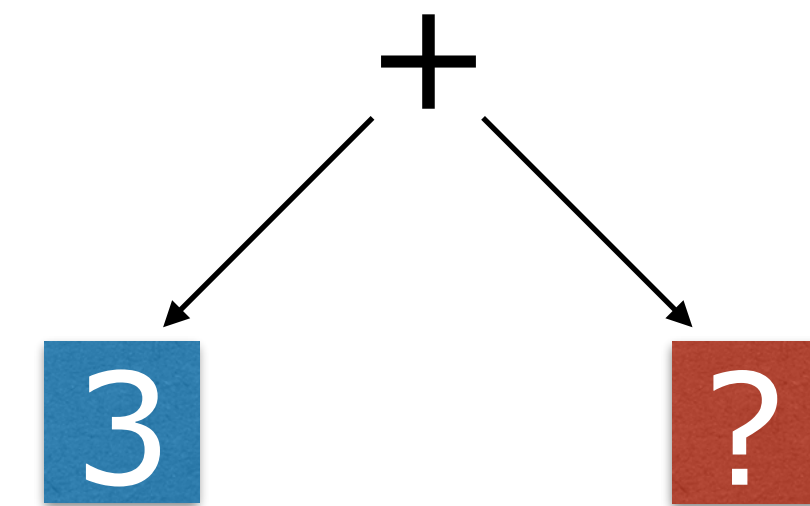
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



$3 + 5 * x$

Grammar of arithmetic expressions

Revised grammar:

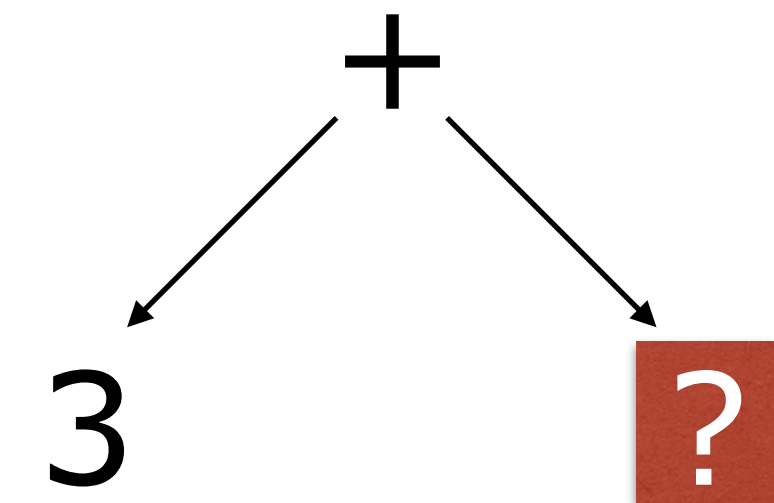
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * x

Grammar of arithmetic expressions

Revised grammar:

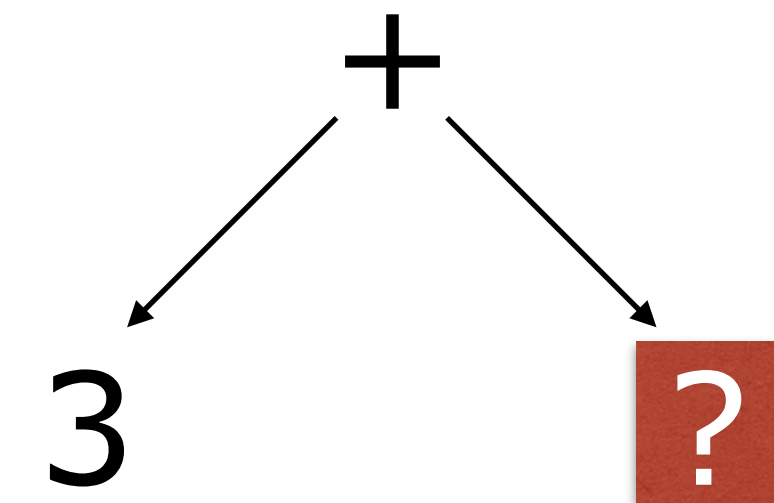
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid \boxed{P}$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * x

Grammar of arithmetic expressions

Revised grammar:

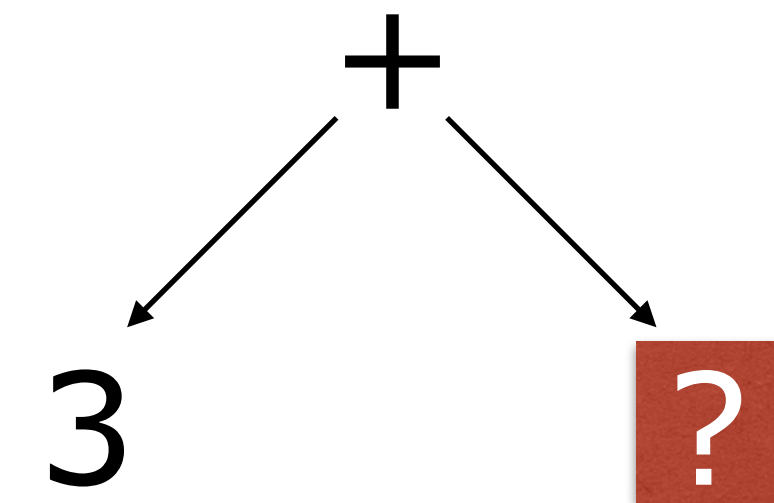
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



$3 + 5 * x$

Grammar of arithmetic expressions

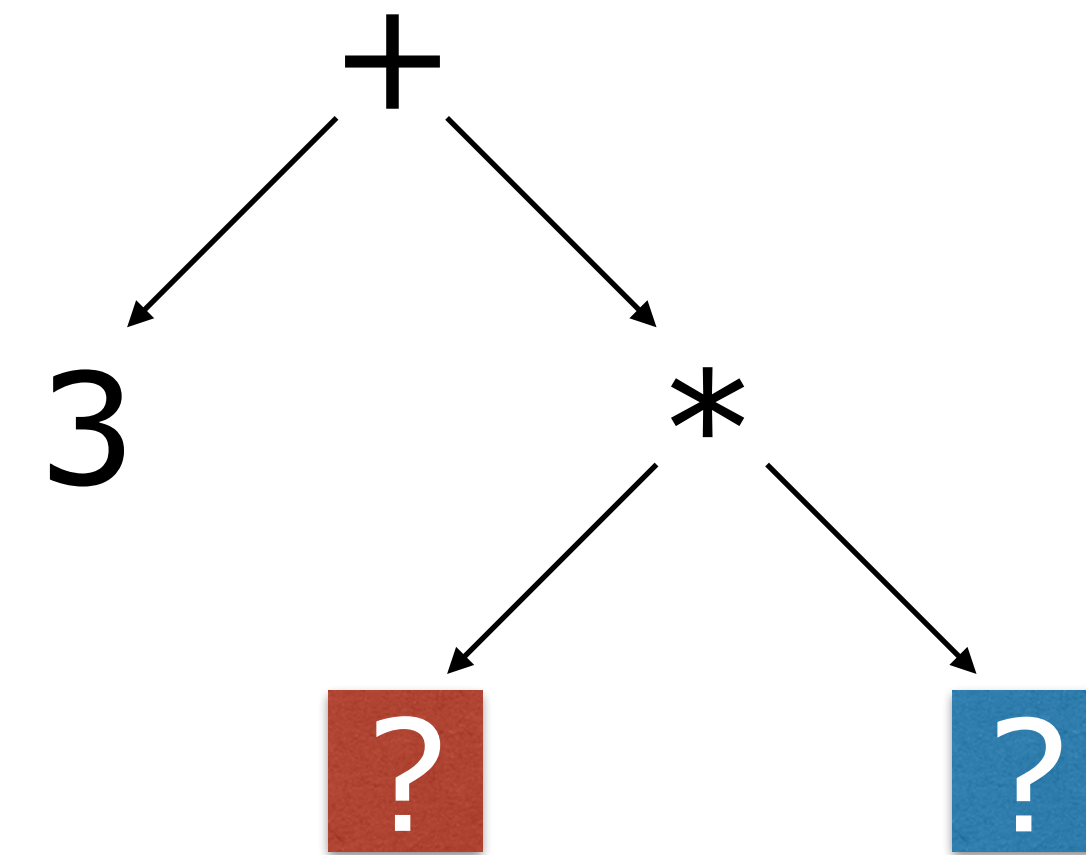
Revised grammar:

$N ::= \langle \textit{integers} \rangle$
 $V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= \boxed{P} * \boxed{P} \mid A$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * x

Grammar of arithmetic expressions

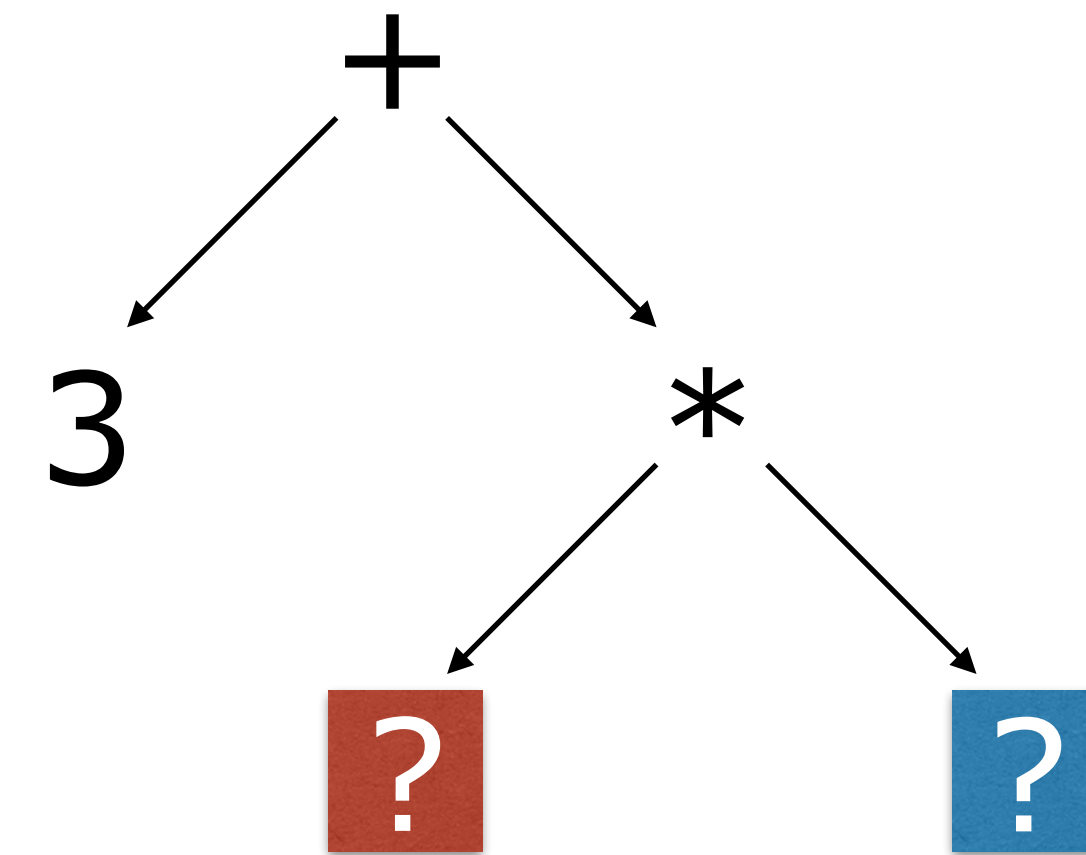
Revised grammar:

$N ::= \langle \textit{integers} \rangle$
 $V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid \boxed{A}$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * x

Grammar of arithmetic expressions

Revised grammar:

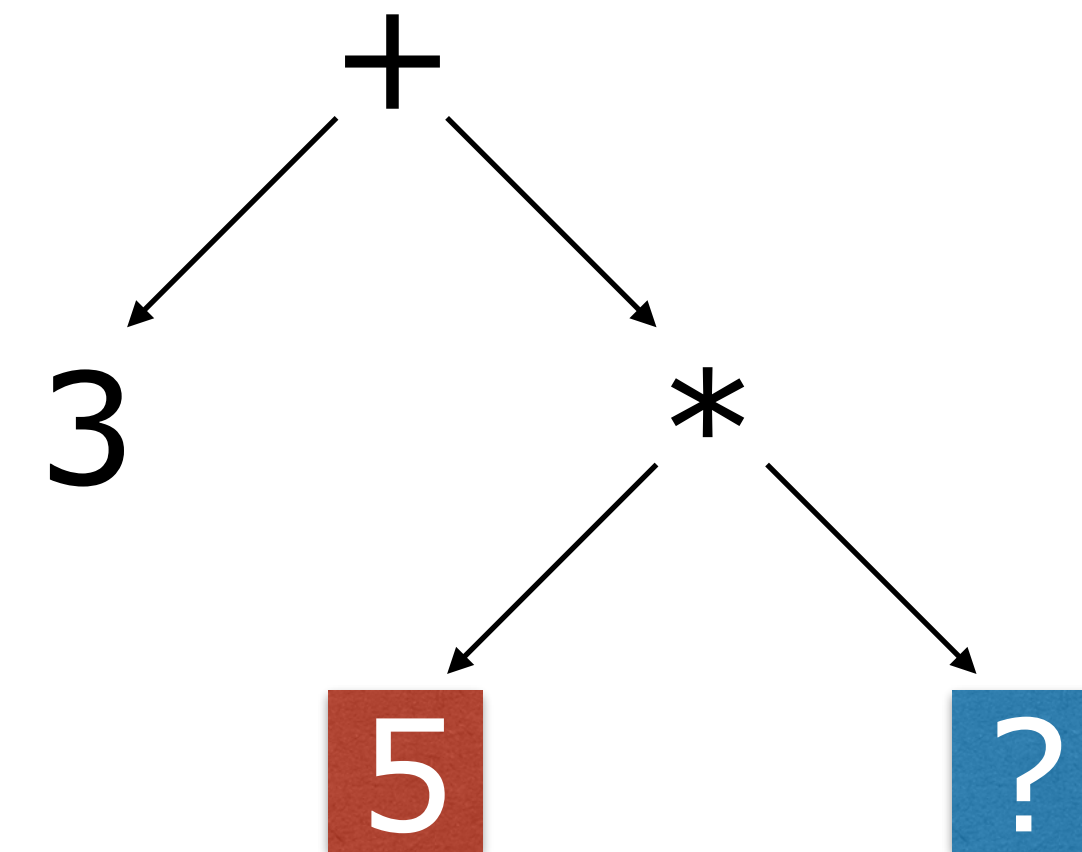
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * x

Grammar of arithmetic expressions

Revised grammar:

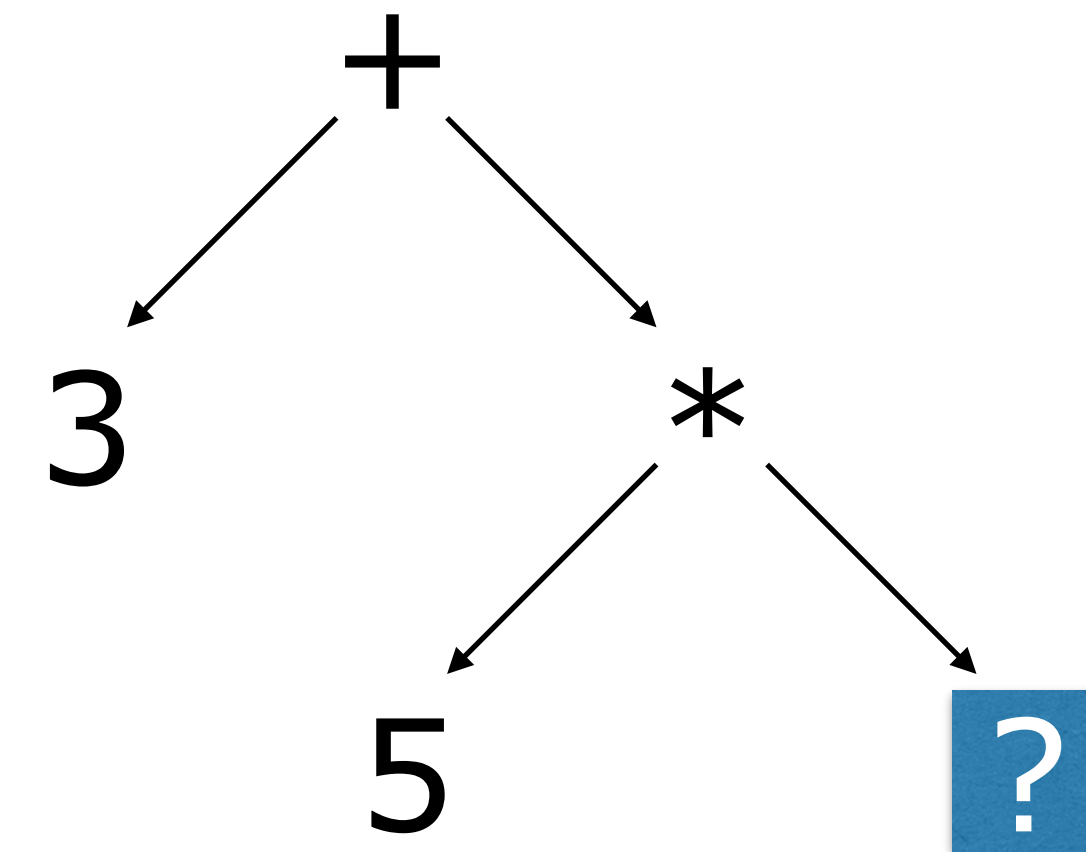
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid \text{A}$

$A ::= N \mid V \mid - A \mid (T)$



3 + 5 * X

Grammar of arithmetic expressions

Revised grammar:

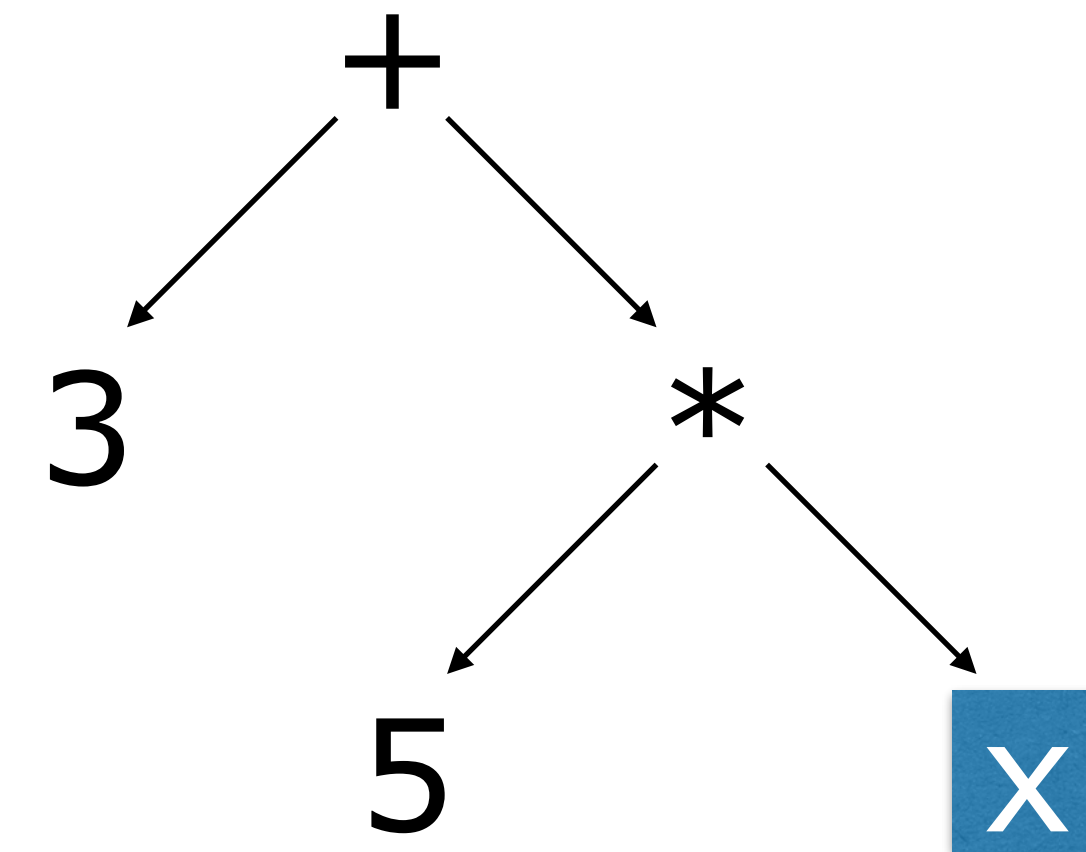
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid \textcolor{blue}{V} \mid - A \mid (T)$



$3 + 5 * \textcolor{blue}{X}$

Grammar of arithmetic expressions

Revised grammar:

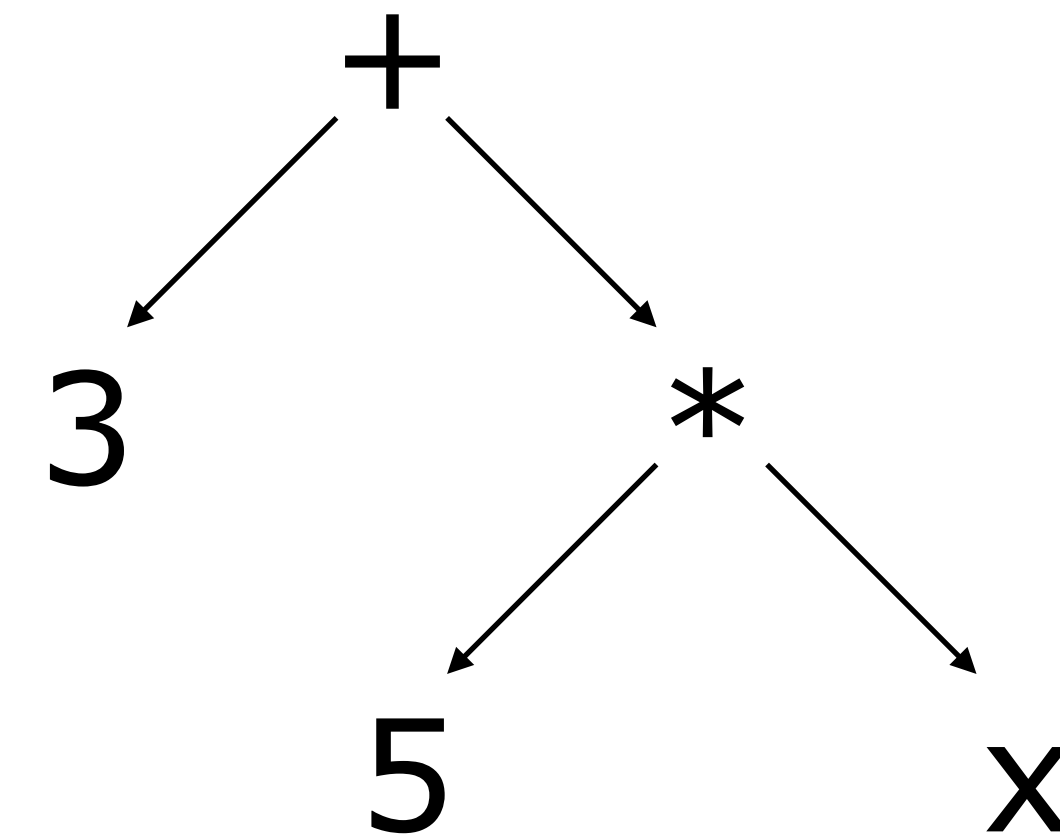
$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$



$3 + 5 * x$

Left-recursive grammars

Revised grammar:

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= T + T \mid P$

$P ::= P * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$

- Note: To pick the right rules to use, we had to look ahead in the input.

Left-recursive grammars

Revised grammar:

$$N ::= \langle \textit{integers} \rangle$$
$$V ::= \langle \textit{variables} \rangle$$
$$T ::= T + T \mid P$$
$$P ::= P * P \mid A$$
$$A ::= N \mid V \mid - A \mid (T)$$

- Note: To pick the right rules to use, we had to look ahead in the input.
- We want to create grammars so that the right rule can be picked by only reading the next character of the input.

Left-recursive grammars

Revised grammar:

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= \boxed{T} + T \mid P$

$P ::= \boxed{P} * P \mid A$

$A ::= N \mid V \mid - A \mid (T)$

The **problem** with our grammar:

A **left-recursive** parser is recursive on the leftmost symbol

By just looking at the next symbol we cannot know which rule to pick

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$$N ::= \langle \textit{integers} \rangle$$
$$V ::= \langle \textit{variables} \rangle$$
$$T ::= T + T \mid P$$
$$P ::= P * P \mid A$$
$$A ::= N \mid V \\ \quad \mid - A \mid (T)$$

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$$\begin{aligned} N &::= \langle \textit{integers} \rangle \\ V &::= \langle \textit{variables} \rangle \end{aligned}$$

$$T ::= T + T \mid P$$

$$P ::= P * P \mid A$$

$$\begin{aligned} A &::= N \mid V \\ &\quad \mid - A \mid (T) \end{aligned}$$

Non-left-recursive grammar

$$\begin{aligned} N &::= \langle \textit{integers} \rangle \\ V &::= \langle \textit{variables} \rangle \end{aligned}$$

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$\begin{aligned} A &::= N \mid V \\ &\quad \mid - A \mid (T) \end{aligned}$$

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$$\begin{aligned} N &::= \langle \textit{integers} \rangle \\ V &::= \langle \textit{variables} \rangle \end{aligned}$$

$$T ::= T + T \mid P$$

$$P ::= P * P \mid A$$

$$\begin{aligned} A &::= N & \mid & V \\ & & \mid & - A & \mid & (T) \end{aligned}$$

Non-left-recursive grammar

$$\begin{aligned} N &::= \langle \textit{integers} \rangle \\ V &::= \langle \textit{variables} \rangle \end{aligned}$$

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$\begin{aligned} A &::= N & \mid & V \\ & & \mid & - A & \mid & (T) \end{aligned}$$

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$N ::= \langle \textit{integers} \rangle$

V

We only need the next
input symbol to choose a
path through the grammar

P

A

$| - A | (T)$

Non-left-recursive grammar

$N ::= \langle \textit{integers} \rangle$

$V ::= \langle \textit{variables} \rangle$

$T ::= P + T \mid P$

$P ::= A * P \mid A$

$A ::= N \mid V$
 $\quad \quad | - A \mid (T)$

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$N ::= \langle \textit{integers} \rangle$

V

T

P

A

We only need the next input symbol to choose a path through the grammar

| - A | (T)

Non-left-recursive grammar

$N ::= \langle \textit{integers} \rangle$

Moreover: left-recursive grammars have the problem that parsing might get stuck in an infinite loop!

| - A | (T)

Remove left recursions

It is relatively straightforward to remove left recursions.

Left-recursive grammar

$$\begin{aligned} N &::= \langle \textit{integers} \rangle \\ V &::= \langle \textit{variables} \rangle \end{aligned}$$

$$T ::= T + T \mid P$$

$$P ::= P * P \mid A$$

$$\begin{aligned} A &::= N \mid V \\ &\quad \mid - A \mid (T) \end{aligned}$$

Non-left-recursive grammar

$$N ::= \langle \textit{integers} \rangle$$

Moreover: left-recursive grammars have the problem that parsing might get stuck in an infinite loop!

$$\mid - A \mid (T)$$

Summary

- We use **context-free grammars** to describe languages and data formats.
- **Abstract syntax trees** represent data described by a context-free grammar.
- A **parser** turns unstructured data (e.g. text or binary) into abstract syntax trees.
- We have to revise grammars in order to account for **operator precedence**.
- We have to revise grammars to avoid **left-recursion**.

Part II

Implementing a Parser

Implementing a parser

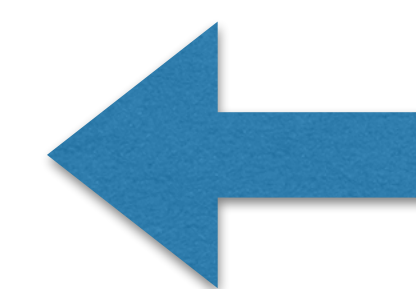
There are 3 different ways to implement a parser:

1. Knuckle down and implement it **from scratch**.
2. Use a **parser generator**, which takes a grammar as input and generates the source code for a parser.
3. Use a library for implementing a parser (called **parser combinator** library).

Implementing a parser

There are 3 different ways to implement a parser:

1. Knuckle down and implement it **from scratch**.
2. Use a **parser generator**, which takes a grammar as input and generates the source code for a parser.
3. Use a library for implementing a parser (called **parser combinator** library).



This is
where FP
shines

Parser combinators

What is the idea behind parser combinators?

- Small parsers for simple things
- Combinators that compose parsers to larger parsers
- Parsers are first-class: can be passed as arguments

Parser combinators

What is the idea behind parser combinators?

- Small parsers for simple things
(e.g. a parser just for parsing integers)
- Combinators that compose parsers to larger parsers
- Parsers are first-class: can be passed as arguments

Parser combinators

What is the idea behind parser combinators?

- Small parsers for simple things
(e.g. a parser just for parsing integers)
- Combinators that compose parsers to larger parsers
(e.g. parser for integers + parser for operator symbols
= parser for arithmetic expressions)
- Parsers are first-class: can be passed as arguments

Parser combinators

What is the idea behind parser combinators?

- Small parsers for simple things
(e.g. a parser just for parsing integers)
- Combinators that compose parsers to larger parsers
(e.g. parser for integers + parser for operator symbols
= parser for arithmetic expressions)
- Parsers are first-class: can be passed as arguments
(e.g. a function that takes a parser and repeats it)

Parser Combinator Libraries

FParsec

- An **industry-grade** parser combinator library
- It is **efficient**, but its implementation is **complicated**.

JParsec

- **Much simpler** library, largely compatible with FParsec.
- We are going to look at this simpler library to understand **how parser combinators work**.

Plan for the rest of the lecture

Goal: Build a complete parser for the arithmetic expression language.

- **First:** Use parser combinators (as black boxes) to build the parser.
- **Afterwards:** Look into how these parser combinators are implemented.

Parser combinators

Parsers are F# values of the following type

`Parser<'a>`

Parsers turn (unstructured) text
into (structured) data of type 'a

Parser combinators

Parsers are F# values of the following type

`Parser<'a>`

Output type of the parser

Parser combinators

Parsers are F# values of the following type

`Parser<'a>`

We can run a parser on a string:

```
run : Parser<'a> -> string ->  
      Result<'a, string>
```

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

- Goal: Write parsers for T, P and A
- All parsers have type `Parser<aexp>`

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```


Parser for arithmetic expressions

$$\begin{aligned}
 T &::= P + T \mid P \\
 P &::= A * P \mid A \\
 A &::= N \mid V \\
 &\quad \mid -A \mid (T)
 \end{aligned}$$

- Goal: Write parsers for T, P and A
- All parsers have type `Parser<aexp>`
- First step: Write parsers for the right-hand sides of the productions rules.

```

type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
  
```


Parser for arithmetic expressions

$$\begin{aligned}
 T &::= P + T \mid P \\
 P &::= A * P \mid A \\
 A &::= N \mid V \\
 &\quad \mid -A \mid (T)
 \end{aligned}$$

```

type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
  
```

- Goal: Write parsers for T, P and A
- All parsers have type Parser<aexp>
- First step: Write parsers for the right-hand sides of the productions rules.
- Afterwards: Put it all together.

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

```
let pN = pint |>> N
```

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

This is the parser for aexp of integers

It uses the following integer parser
from the parser combinator library:

```
pint : Parser<int>
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

```
let pN = pint |>> N
```

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

Mapping function for parsers

$$(|>>) : \text{Parser}<'a> \rightarrow ('a \rightarrow 'b) \rightarrow \text{Parser}<'b>$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

```
let pN = pint |>> N
```

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

Mapping function for parsers

$$(|>>) : \text{Parser}<'a> \rightarrow ('a \rightarrow 'b) \\ \rightarrow \text{Parser}<'b>$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
pint : Parser<int>
```

```
N : int -> aexp
```

```
pN : Parser<aexp>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \mid -A \mid (T)$$

```
let pN  = pint |>> N
let pV  = pid  |>> V
```

Parser for aexp of variables

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
pid : Parser<string>
```

```
V : string -> aexp
```

```
pV : Parser<aexp>
```

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \mid -A \mid (T)$$

```
let pN = pint |>> N
let pV = pid  |>> V
```

Parser for aexp of variables

```
pid : Parser<string>
```

```
let pid = many1 pletter |>> charListToStr
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \mid -A \mid (T)$$

```
let pN  = pint |>> N
let pV  = pid  |>> V
```

Parser for aexp of variables

```
pid : Parser<string>
```

```
let pid = many1 pletter |>> charListToStr
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
charListToStr : char list -> string
```

```
pletter : Parser<char>
```

```
many1 : Parser<'a> -> Parser <'a list>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V$$

parses a letter
(see assignment)

```
type aexp =
  N of int
```

many1 p
repeats p one
or more times

aexp
aexp

```
let pN = pint |>> N
let pV = pid |>> V
```

Parser for aexp of variables

```
pid : Parser<string>
```

```
let pid = many1 pletter |>> charListToStr
```

```
charListToStr : char list -> string
```

```
pletter : Parser<char>
```

```
many1 : Parser<'a> -> Parser <'a list>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. pA) |>> Neg
```

```
pchar : char -> Parser<char>
```

```
(>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'b>
```

```
pA : Parser<aexp>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. pA) |>> Neg
```

```
pchar : char -> Parser<char>
```

```
(>>.) : Parser<'a> ->
      Parser<'b> ->
      Parser<'b>
```

```
pA : Parser<aexp>
```

Assume for now
that the parser for
A is already
defined.

Neg of aexp

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
pchar : char -> Parser<char>
```

```
(>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'b>
```

```
pA : Parser<aexp>
```

```
spaces : Parser<char list>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
pchar : char -> Parser<char>
```

```
(>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'b>
```

```
pA : Parser<aexp>
```

```
spaces : Parser<char list>
```

```
type aexp =
```

parses zero or
more white space
characters
(see assignment)

Parser for arithmetic expressions

$T ::= P + T \mid P$

$P ::= A * P \mid A$

$A ::= N \mid V$
 $\quad \mid -A \mid (T)$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
(>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'b>
```

```
(.>>) : Parser<'a> ->
        Parser<'b> ->
        Parser<'a>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(') pT (pchar ')')
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

pT : Parser<aexp>

between : Parser <'a> ->

Parser <'b> ->

Parser <'c> ->

Parser <'b>

between l p r = l >>. p .>> r

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT
                  (spaces >>. pchar ')')
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

pT : Parser<aexp>

```
between : Parser <'a> ->
         Parser <'b> ->
         Parser <'c> ->
         Parser <'b>
```

```
between l p r = l >>. p .>> r
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT
                  (spaces >>. pchar ')')
```

```
let pAdd = (pP .>> pchar '+' .>>. pT) |>> Add
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
(.>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'a * 'b>
```

Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT
                  (spaces >>. pchar ')')
```

```
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces)
            .>>. pT) |>> Add
```

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
(.>>.) : Parser<'a> ->
        Parser<'b> ->
        Parser<'a * 'b>
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT
                  (spaces >>. pchar ')')
```

```
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces)
            .>>. pT) |>> Add
```

```
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces)
            .>>. pP) |>> Mul
```


Parser for arithmetic expressions

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
type aexp =
| N of int
| V of string
| Add of aexp * aexp
| Mul of aexp * aexp
| Neg of aexp
```

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT
                  (spaces >>. pchar ')')
```

```
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces)
            .>>. pT) |>> Add
```

```
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces)
            .>>. pP) |>> Mul
```

We are almost done.
What do we do about pA, pT, and pP?

Putting it all together

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

```
let rec pT = (pP .>> (spaces .>>. pchar '+' .>>. spaces) .>>. pT |>> Add)
              <|> pP
```

```
and pP = ...
```

```
and pA = ...
```

$(\langle | \rangle) : \text{Parser } \langle 'a \rangle \rightarrow \text{Parser } \langle 'a \rangle \rightarrow \text{Parser } \langle 'a \rangle$

Putting it all together

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

- F# will not accept this definition! (Boooo!)
- Only functions may be defined recursively!
(otherwise the right-hand is evaluated immediately)
- We have to use some trickery instead.

```
let rec pT = (pP .>> (spaces .>>. pchar '+' .>>. spaces) .>>. pT |>> Add)
               <|> pP
```

```
and pP = ...
```

```
and pA = ...
```

$(\langle | \rangle) : \text{Parser } \langle 'a \rangle \rightarrow \text{Parser } \langle 'a \rangle \rightarrow \text{Parser } \langle 'a \rangle$

Recursive production rules

```
let pT, tref = createParserForwardedToRef<aexp>()
```

...

```
do tref := (pP .>> (spaces >>. pchar '+' >>. spaces)  
             .>>. pT |>> Add) <|> pP
```

- We create a dummy value for pT and a reference to it:

```
createParserForwardedToRef : unit -> Parser<'a> * Parser<'a> ref
```

- pT can be used in declarations immediately
- but the reference **must** be set before the parser is used

The complete parser definition

```
let pT, tref = createParserForwardedToRef<aexp>()  
let pP, pref = createParserForwardedToRef<aexp>()  
let pA, aref = createParserForwardedToRef<aexp>()
```

$$T ::= P + T \mid P$$
$$P ::= A * P \mid A$$
$$A ::= N \mid V \\ \mid -A \mid (T)$$

The complete parser definition

```
let pT, tref = createParserForwardedToRef<aexp>()
let pP, pref = createParserForwardedToRef<aexp>()
let pA, aref = createParserForwardedToRef<aexp>()
```

```
let pN = pint |>> N
let pV = pid |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces) .>>. pT) |>> Add
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces) >>. pP) |>> Mul
```

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

The complete parser definition

```
let pT, tref = createParserForwardedToRef<aexp>()
let pP, pref = createParserForwardedToRef<aexp>()
let pA, aref = createParserForwardedToRef<aexp>()
```

```
let pN = pint |>> N
let pV = pid |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces) .>>. pT) |>> Add
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces) >>. pP) |>> Mul
```

```
do tref := pAdd <|> pP
do pref := pMul <|> pA
do aref := pNeg <|> pPar <|> pN <|> pV
```

$$T ::= P + T \mid P$$

$$P ::= A * P \mid A$$

$$A ::= N \mid V \\ \mid -A \mid (T)$$

Two improvements

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces) .>>. pT) |>> Add
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces) >>. pP) |>> Mul
```

Two improvements

Use combinators for parsing binary/unary operators.
Implementing these combinators is part of the assignment

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
let pNeg = (pchar '-' >>. spaces >>. pA) |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = (pP .>> (spaces >>. pchar '+' >>. spaces) .>>. pT) |>> Add
let pMul = (pA .>> (spaces >>. pchar '*' >>. spaces) >>. pP) |>> Mul
```


Two improvements

Use combinators for parsing binary/unary operators.
Implementing these combinators is part of the assignment

```
let pN    = pint |>> N
let pV    = pid  |>> V
```

```
unop : Parser<'a> -> Parser<'b>
      -> Parser<'b>
```

```
let pNeg = unop (pchar '-') pA |>> Neg
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
```

```
let pAdd = binop (pchar '+') pP pT |>> Add
```

```
let pMul = binop (pchar '*') pA pP |>> Mul
```

```
binop : Parser<'a> -> Parser<'b> -> Parser<'c>
      -> Parser<'b * 'c>
```

Two improvements

Attach labels to each parser to improve error messages:

```
<?> : Parser<'a> -> string -> Parser<'a>
```

```
let pN    = pint |>> N <?> "integer"
let pV    = pid  |>> V <?> "variable"
```

```
let pNeg = unop (pchar '-') pA |>> Neg <?> "negated expression"
```

```
let pPar = between (pchar '(' >>. spaces) pT (spaces >>. pchar ')')
let pAdd = binop (pchar '+') pP pT |>> Add <?> "addition"
let pMul = binop (pchar '*') pA pP |>> Mul <?> "multiplication"
```


Summary

- There are different approaches to writing parsers.
- We used a **parser combinator** library.
- A parser is built by writing **small parsers** for parts of the language.
- Parsers are **combined** to form larger parsers **using parser combinators**.
- In F# we have to be careful about **recursive parsers**.

Part III

Parser Library

Implementing the Parser Library

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
type ParseError = string
```

```
type Result<'a, 'b> =  
| OK      of 'a  
| Error  of 'b
```

Implementing the Parser Library

input string

success

failure

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
type ParseError = string
```

```
type Result<'a, 'b> =  
| OK      of 'a  
| Error  of 'b
```


Implementing the Parser Library

input string success failure

```

type Parser<'a> = string -> Result<'a * string, ParseError>
  
```

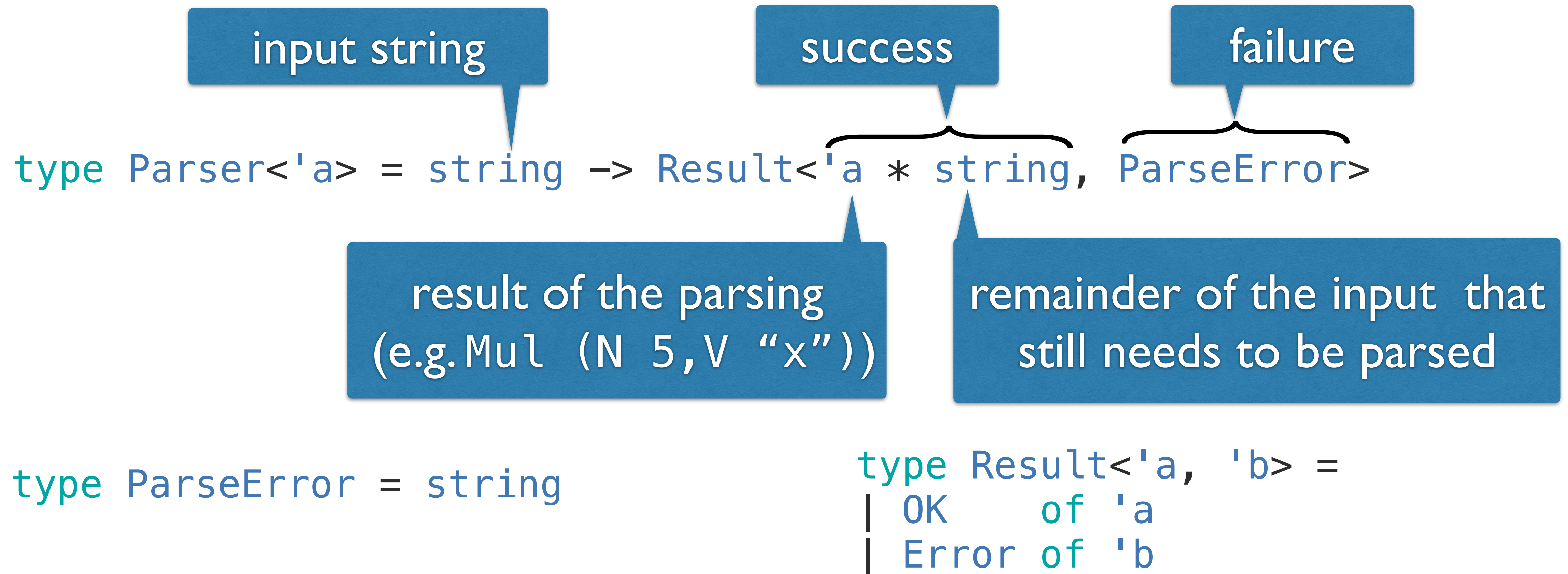
result of the parsing
 (e.g. `Mul (N 5, V "x")`)

```

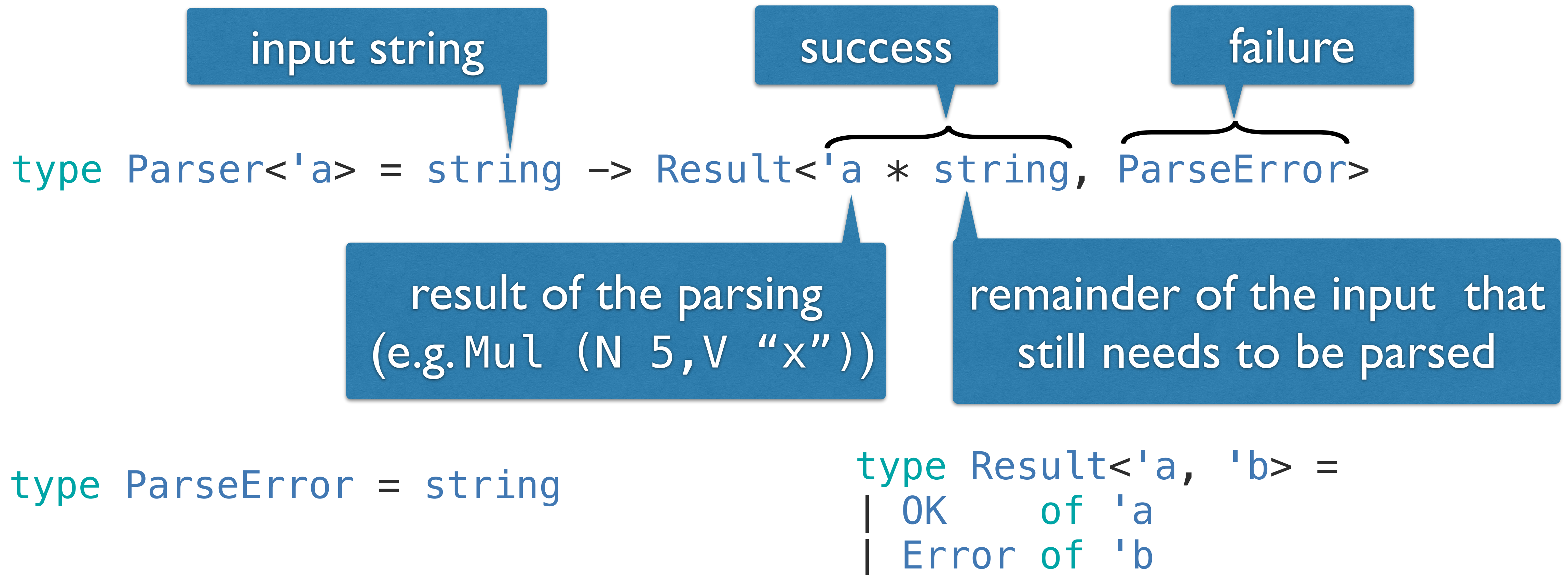
type ParseError = string

type Result<'a, 'b> =
  | OK      of 'a
  | Error  of 'b
  
```

Implementing the Parser Library



Implementing the Parser Library



NB: this is a slight simplification that does not support `<?>`

Implementing pchar

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
let pchar (ch : char) : Parser<char> = fun str ->  
    if str.Length = 0 then  
        Error "No more input"  
    else  
        let first = str.[0]  
        if first = ch then  
            let remaining = str.[1..]  
            Ok (first, remaining)  
        else  
            Error (sprintf "Expecting '%c'. Got '%c'" ch first)
```

Implementing .>>.

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
(.>>.) : Parser<'a> -> Parser<'b> -> Parser<'a * 'b>
```

```
let (.>>.) p1 p2 = fun inp ->
  match p1 inp with
  | Error err      -> Error err
  | Ok (val1, rem1) ->
    match p2 rem1 with
    | Error err      -> Error err
    | Ok (val2, rem2) -> Ok ((val1, val2), rem2)
```

Implementing $|>>$

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
(|>>) : Parser<'a> -> ('a -> 'b) -> Parser<'b>
```

```
let (|>>) p f = fun inp ->  
  match p inp with  
  | Ok (value, rem) -> Ok (f value, rem)  
  | Error err       -> Error err
```


Implementing $>>.$ and $.>>$

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
(.>>.) : Parser<'a> -> Parser<'b> -> Parser<'a * 'b>
```

```
(.>>)   : Parser<'a> -> Parser<'b> -> Parser<'a>
```

```
(>>.)   : Parser<'a> -> Parser<'b> -> Parser<'b>
```

```
let (.>>) p1 p2 = (p1 .>>. p2) |>> fst
```

```
let (>>.) p1 p2 = (p1 .>>. p2) |>> snd
```

Implementing `<|>`

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
(<|>) : Parser<'a> -> Parser<'a> -> Parser<'a>
```

```
let (<|>) p1 p2 = fun inp ->  
  match p1 inp with  
  | Ok result -> Ok result  
  | Error err -> p2 inp
```

Implementing many

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
many : Parser<'a> -> Parser <'a list>
```

```
repeat : Parser<'a> -> string -> 'a list * string
```

```
let rec repeat p inp =  
  match p inp with  
  | Error err          -> ([], inp)  
  | Ok (value, rem1) ->  
    let (values, rem2) = repeat p rem1  
    (value::values, rem2)
```

helper function

```
let many p = fun inp -> Ok (repeat p inp)
```


Implementing many1

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

```
many : Parser<'a> -> Parser <'a list>
```

```
many1 : Parser<'a> -> Parser <'a list>
```

similar to many, but with at least 1 repetition

```
let many1 p = (p .>>. many p) |>> (fun (v,vs) -> v :: vs)
```

Generalise pchar

Recall: pchar : char → Parser<char>

Generalise pchar

Recall: pchar : char -> Parser<char>

```
let satisfy (p : char -> bool) : Parser<char> = fun str ->
  if str.Length = 0 then
    Error "No more input"
  else
    let first = str.[0]
    if p first then
      let remaining = str.[1..]
      Ok (first, remaining)
    else
      Error (sprintf "Unexpected '%c'" first)
```


Generalise pchar

Recall: `pchar : char -> Parser<char>`

```
let satisfy (p : char -> bool) : Parser<char> = fun str ->
  if str.Length = 0 then
    Error "No more input"
  else
    let first = str.[0]
    if p first then
      let remaining = str.[1..]
      Ok (first, remaining)
    else
      Error (sprintf "Unexpected '%c'" first)

let pchar c = satisfy (fun x -> x = c)
```

Refining the Parser type

```
type Parser<'a> = string -> Result<'a * string, ParseError>
```

Replace string type with this type

```
type TextInputState = {  
    lines      : string[]  
    position   : Position  
}  
  
where  
  
type Position = {  
    line      : int  
    column    : int  
}
```

- The parser will only change the position (more efficient)
- We can provide better error messages
 - We can give the position where the parser failed!

Implementing <?>

Our type definition do not support <?> yet.

```
type Parser<'a> = TextInputState ->  
                  Result<'a * TextInputState, ParseError>
```

```
type ParseError = string
```

```
type Result<'a, 'b> =  
  | Ok      of 'a  
  | Error   of 'b
```


Implementing <?>

We extend the ParseError type with a label field

```
type Parser<'a> = TextInputState ->  
                  Result<'a * TextInputState, ParseError>
```

```
type ParseError = {  
    label : string  
    input : TextInputState  
    error : string }
```

```
type Result<'a, 'b> =  
    | Ok      of 'a  
    | Error   of 'b
```

Implementing <?>

We extend the ParseError type with a label field

```
type Parser<'a> = TextInputState ->
                  Result<'a * TextInputState, ParseError>
```

```
type ParseError = {
  label : string
  input : TextInputState
  error : string }
```

what did the parser expect
(e.g. “integer”)

where did the parser fail

```
type Result<'a, 'b> =
  | Ok      of 'a
  | Error  of 'b
```

Implementing <?>

```
type Parser<'a> = TextInputState ->  
                  Result<'a * TextInputState, ParseError>
```

```
type ParseError = {  
    label : string  
    input : TextInputState  
    error : string }
```

```
(<?>) : Parser<'a> -> string -> Parser<'a>
```

```
let (<?>) p l = fun input ->  
    match p input with  
    | Ok s    -> Ok s  
    | Error pe -> Error { pe with label = l }
```


Running a parser

```
type Parser<'a> = TextInputState ->  
                Result<'a * TextInputState, ParseError>  
run : Parser<'a> -> string -> Result<'a, string>
```

Running a parser

```
type Parser<'a> = TextInputState ->
                  Result<'a * TextInputState, ParseError>

run : Parser<'a> -> string -> Result<'a, string>

let run p str =
  match p (inputStr str) with
  | Ok (value, _) -> Ok value
  | Error err      -> Error (errorStr err)
```

Running a parser

```
type Parser<'a> = TextInputState ->
                  Result<'a * TextInputState, ParseError>
run : Parser<'a> -> string -> Result<'a, string>

let run p str =
    match p (inputStr str) with
    | Ok (value, _) -> Ok value
    | Error err     -> Error (errorStr err)
```

inputStr : string -> TextInputState

Running a parser

```
type Parser<'a> = TextInputState ->  
                  Result<'a * TextInputState, ParseError>
```

```
run : Parser<'a> -> string -> Result<'a, string>
```

```
let run p str =  
  match p (inputStr str) with  
  | Ok (value, _) -> Ok value  
  | Error err      -> Error (errorStr err)
```

```
errorStr : ParseError -> string
```

Running a parser

```
type Parser<'a> = TextInputState ->
                Result<'a * TextInputState, ParseError>
```

```
run : Parser<'a> -> string -> Result<'a, string>
```

```
let run p str =
    match p (inputStr : string -> TextInputState) str with
    | Ok (value, _) -> Ok value
    | Error err      -> Error (errorStr err)
```

```
errorStr : ParseError -> string
```

```
let errorStr err = sprintf "Error parsing %s at %A\n%s"
                        err.label err.input.position err.error
```

One last refinement

```
type Parser<'a> = {  
  pfun    : TextInputState ->  
           Result<'a * TextInputState, ParseError>  
  label  : string }
```

Label is attached to parser itself as well.

One last refinement

```
type Parser<'a> = {  
  pfun    : TextInputState ->  
           Result<'a * TextInputState, ParseError>  
  label   : string }
```

Label is attached to parser itself as well.

This allows us to update the label in the combinators, e.g.

```
let (.>>.) p1 p2 = {  
  pfun = fun inp -> ... // defined as before  
  label = "unknown" }  
  <?> sprintf "(%s .>>. %s)" (p1.label) (p2.label)
```

Summary

- **A parser is a function** turning input into a **partial result + remaining input**
- **simple parsers** can be constructed by simply inspecting the input
- **parser combinators** take parsers as input and construct a new parser by distributing the input to the old parsers
- **error messages** can be adjusted by attaching **labels** to parsers